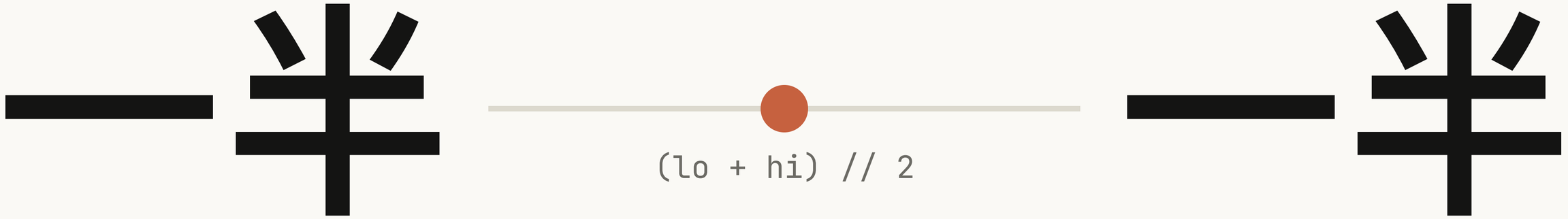
# LESSON 26




## 二分查找：每次排除一半

学完你能让电脑猜 1 到 100 的数最多 7 次就猜中，还能在排好序的数里飞快找到一个。

# 猜 1 到 100 的数，怎么猜最快?

上节课的办法：从 1 猜起

**最多猜 100 次**

先猜中间的 50

**一下去掉一半**

猜 50，对方说“大了”，51 到 100 就都不用想了。每次都猜剩下的正中间，每次都去掉一半。这叫二分查找。它有个前提：数要排好顺序，才能从“大了、小了”知道往哪边找。

# 猜一猜：每次都猜中间，最多要猜几次?

剩 100 → 50 → 25 → 12 → 6 → 3 → 1

## 最多 7 次

猜一次没中，剩下的不到一半。猜 6 次以后最多剩 1 个，第 7 次一定中。
从 1 猜起最多要 100 次，二分只要 7 次。数越多，差得越远：1000 个数也只要 10 次。

# 中间怎么算

mid.py

```
1 lo = 1
2 hi = 100
3 mid = (lo + hi) // 2
4 print(mid)
```

两头加起来除以 2 就是中间。用 // 整除，101 // 2 是 50，结果还是整数，正好拿来猜。

| | |
|---|---|
| lo | 1 |
| hi | 100 |
| mid | 50 |

# 小测：lo = 51，hi = 100，mid 是几？

1 75

2 75.5

3 76

4 50

答对了！51 + 100 是 151，151 // 2 是 75。

# 例题：电脑来猜你的数

guess.py

```
secret = int(input())
lo = 1
hi = 100
guess = 0
while guess != secret:
    guess = (lo + hi) // 2
    print(guess)
    if guess < secret:
        lo = guess + 1
    elif guess > secret:
        hi = guess - 1
```

| | |
|---|---|
| 1–100 | 猜 **50**，大了 |
| 1–49 | 猜 **25**，小了 |
| 26–49 | 猜 **37**，小了 |
| 38–49 | 猜 **43**，大了 |
| 38–42 | 猜 **40**、**41**、**42** |

范围越来越小，第 7 次猜到 42，guess 等于 secret，循环停下。

# 小测：例题里秘密数是 50，电脑猜几次？

1 1 次

2 7 次

3 50 次

4 0 次

答对了！第一次就猜中间 50，正好中了。

# 动手改一改

加一个计数器。

## 最后说出猜了几次

在例题上改：每猜一次，计数加 1。猜中以后，再输出一行“猜了几次”。秘密数是 1 到 100 的整数。

| 秘密数 42 | 秘密数 50 | 秘密数 100 |
| --- | --- | --- |
| **猜了 7 次** | **猜了 1 次** | **猜了 7 次** |

把 1 到 100 都试一试，看看有没有超过 7 次的。教室电脑：打开 starter/03_guess_count.py

# # 这节课常见的三种错误

划线的写法只阅读，不要照抄

| | | |
|---|---|---|
| ~~`lo = guess`~~ | **不报错** | 可能一直猜同一个，停不下来 |
| ~~`(lo + hi) / 2`~~ | **不对劲** | `/` 得到小数，猜 `50.5` 这样的数 |
| 数没排好就二分 | **不报错** | 可能找不到明明有的数 |

猜过的数已经不可能了，所以写 `guess + 1` 和 `guess - 1`。用 `1` 和 `100` 两头试一试。

# 找错：秘密数是 100，一直猜 99

bug.py（只阅读）

```
1  secret = int(input())
2  lo = 1
3  hi = 100
4  guess = 0
5  while guess != secret:
6      guess = (lo + hi) // 2
7      print(guess)
8      if guess < secret:
9          lo = guess
10     elif guess > secret:
11         hi = guess - 1
```

输出 50、75、87、93、96、98、99，然后一直是 99。
点出该改的那一行。

点代码里的一行

# 自己挑战

# 在排好的数里找

图书馆的书按编号从小到大排好了。第一行读入 n（1 到 100 的整数），接下来 n 行是排好的编号，最后一行是要找的编号。
用二分查找，输出两行：它的下标，找不到就输出 -1；一共看了几个数。

依次输入

```
8
3
8
15
21
30
42
57
66
42
```

输出

```
5
2
```

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　这回 lo 和 hi 是下标：lo = 0，hi = n - 1。每次看中间的 a[mid]，看一次计数加 1。

**提示 2**　a[mid] 比要找的小，往右半边找：lo = mid + 1；大了往左半边：hi = mid - 1。

**提示 3**　用 pos = -1 记着下标，找到了就改成 mid。循环条件写 lo <= hi and pos == -1：还有得找，又还没找到。

# 造一个“每次砍一半”的小程序

找一件能对半排除的事，让程序来找。

## 先做到

把例题的范围改成 1 到 1000，看看电脑最多要猜几次。

## 选你喜欢的

翻书找页码：每次翻到中间，说往前翻还是往后翻；猜同桌的身高。

## 还想更厉害

你心里想一个数，程序每猜一次就问你，你输入“大了”或“小了”。

没有标准答案，能运行、你喜欢就好。

# # 和同桌换着玩

## 1

### 比一比

你用嘴猜，同桌的程序来猜，看谁次数少

## 2

### 说一个喜欢的

告诉同桌，哪里想得最巧

## 3

### 提一个想法

1 和最大的那个数，能猜到吗?

听完再改一版，让程序更稳。

# 说得清吗?

| | | |
|---|---|---|
| Q1 | **二分查找为什么快?** | 每次去掉一半，1 到 100 最多 7 次。 |
| Q2 | **用二分有什么前提?** | 数要排好顺序。 |
| Q3 | **为什么是 `guess + 1`，不是 `guess`?** | 猜过的已经不对，不用再留着，不然可能停不下来。 |

# 速查卡

| | |
|---|---|
| `mid = (lo + hi) // 2` | 取中间，用整除 |
| `lo = mid + 1` | 中间太小，去右半边 |
| `hi = mid - 1` | 中间太大，去左半边 |
| `while lo <= hi:` | 还有数没排除，就接着找 |
| **先排好序** | 二分只能用在排好的数上 |

# # 这些后面再学

## 二分答案

答案不在列表里，也能对半猜，比如开平方

## `bisect`

Python 自带的二分工具

## 比一比快慢

数越多，挨个试和二分差得越远

今天学会了每次排除一半。排序和二分常常一起用：先排好，再快快地找。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

回到课程首页

这门课的路线走完了，了不起！